

Delphi App Translation Studio Engineering Guide

Understand the files, components, and release workflow

Version 2 | Last changed 7 October 2026

Contents

How the system is assembled....3
How a project becomes a catalog....3
Where the files live....4
Translation and review states....5
Validation and pack construction....6
The component kit and Delphi project....7
What happens when the application runs....8
Layout and deployment boundaries....8
Backup and recovery responsibilities....9
Engineering release check....9

How the system is assembled

Delphi App Translation Studio has two sides. The Studio is the authoring tool: it discovers application text, keeps editable catalogs, asks a translation provider for drafts, validates each language, and writes offline packs. The Delphi application is the consumer: its DAT components read those packs and apply the selected language while the application runs. The translation provider is not needed by the finished application.

These sides deliberately own different files. The Studio owns its workspace and generated packs. The developer owns the Delphi project, the form design, and the decision to distribute a build. A scan never inserts components into a form or rewrites Pascal, DPR, DPROJ, DFM, or FMX files. The component kit gives RAD Studio the units and design package needed to connect the runtime explicitly.

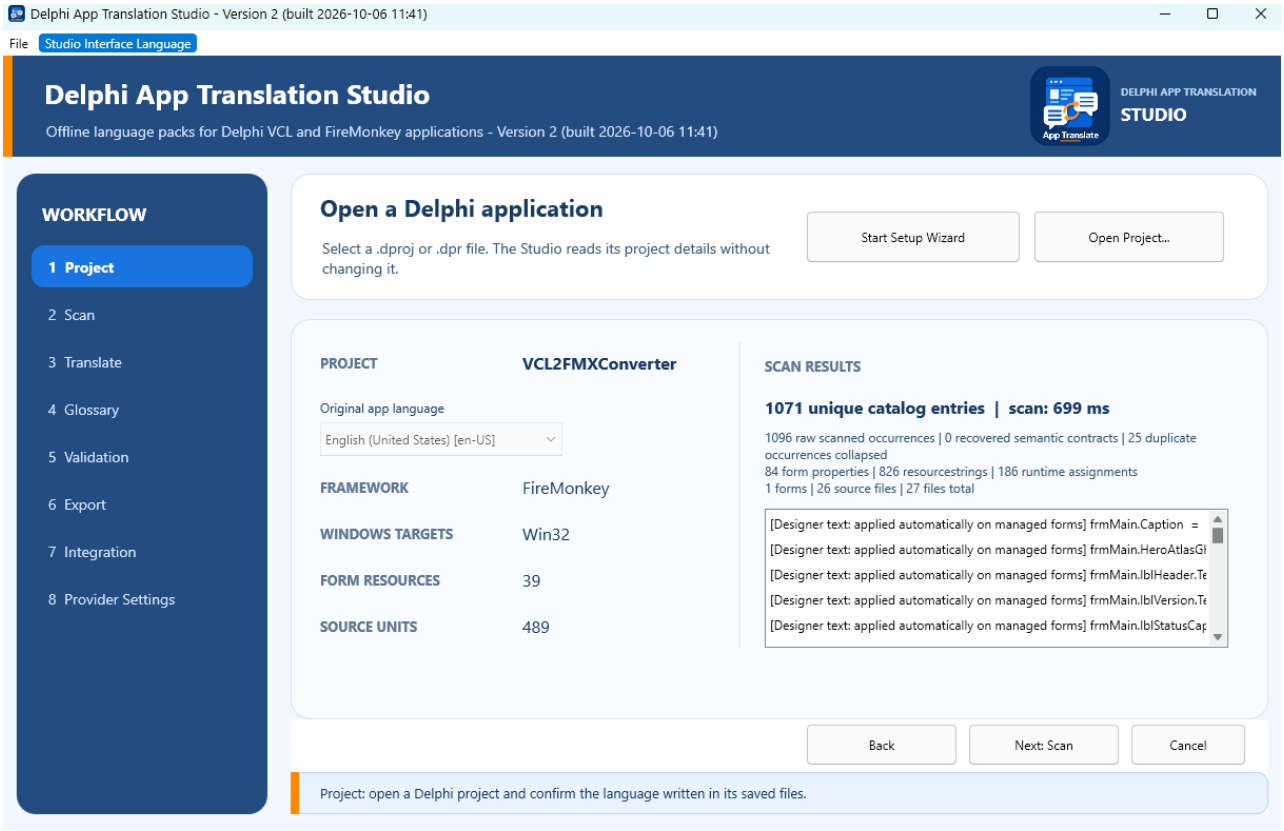
Part	Owns	Hands forward
Scanner	Saved project text and source context	Entries with stable keys
Catalog	Editable translations and review state	Validated language
Exporter	Offline source and target packs	Pack files for deployment
DAT runtime	Pack checks and live language selection	Translated application UI

The key distinction is between a development catalog and a runtime pack. A catalog is the editable record of source text, translation, status, context, and review notes. A runtime pack is the smaller validated JSON file read by the application. A saved catalog is therefore valuable work, but it is not a deployed language.

How a project becomes a catalog

The Studio starts from a saved .dproj or .dpr. Project detection identifies the framework and supported Windows targets. The scanner reads referenced forms and Pascal source, including form properties, resource strings, and supported runtime text assignments. It can read declared external resources. Supplemental project files are considered under scan rules, while bundled dependencies and test-contract files are excluded unless the Delphi project explicitly references them. The scanner records diagnostics for material it cannot safely interpret.

Each scan occurrence receives a key and context: where the text came from, the form or component when known, and whether the runtime can apply it automatically. A form caption is different from a changing data value even if their words look alike. Equivalent occurrences can be collapsed into one catalog entry; that is why the raw occurrence count can exceed the unique-entry count. A duplicate key with conflicting source text is treated as a collision rather than silently choosing one translation.



The project screen retains a scan summary. Unique entries are catalog records; raw occurrences include repeated appearances of the same text.

Rescanning merges the new scan with saved work by key. Unchanged source text keeps its translation. Changed source text is marked for review; newly found text needs translation. An entry absent from the new scan becomes obsolete, unless an explicit stable semantic contract is still supported by the same source literal. The source-language catalog maps each current source string to itself. This keeps the original-language pack current without sending source text to a provider as a same-language request.

Where the files live

The Studio keeps editable work under the current Windows user's local application data in DelphiAppTranslationStudio\Workspaces. A workspace is identified from the project name and project path; an older compatible project workspace can be reused. This prevents two project copies with the same name from automatically sharing translations. The Development folder holds the editable per-language catalogs, Glossaries holds preferred terms, and Deployment stores saved destinations and session information. The Languages folder holds exported packs, grouped by source language.

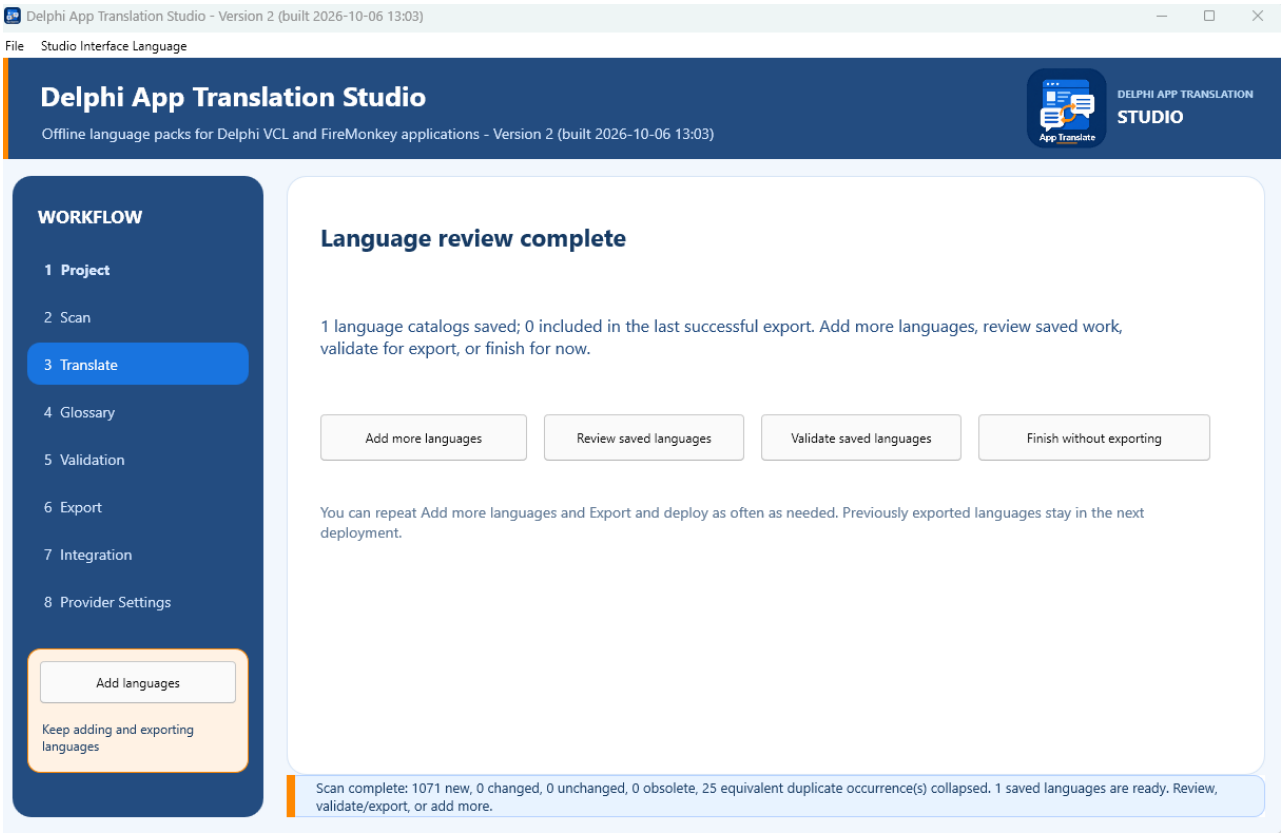
Location	What it contains	Who uses it
Workspace / Development	Editable catalogs	Studio
Workspace / Languages	Exported pack files	Studio
Project / Localization / Languages	Packs for later builds	Build and deploy
EXE folder / Localization / Languages	Packs beside that EXE	Running application

The project-local Localization\Languages folder serves builds and Studio deployment. An executable uses the Localization\Languages folder beside that exact executable. Copying only the EXE to another folder does not copy its packs. The managed dependency under dependencies\DelphiAppTranslation\source is different again: it contains Delphi runtime units, not translated text.

Translation and review states

A new target entry begins as text needing translation. DeepL or Google Cloud Translation can fill it as a draft. The provider can use glossary and context information, but its output is still a proposal. The catalog records its origin and status; a person can edit it, mark it Reviewed, or separately Approve it. These states help the developer see what was checked. They do not by themselves place a pack beside an executable.

The catalog also records how the text reaches the UI. Designer-owned static text can be applied by the language manager on a managed form. A runtime template can be looked up by key when the application formats a message. A dynamic value assembled by application code may need an explicit translation or formatting call; the Studio cannot safely infer every such call from a scan. A warning about unconfirmed runtime wiring means the words may be translated in the catalog but still never appear translated in the running application.

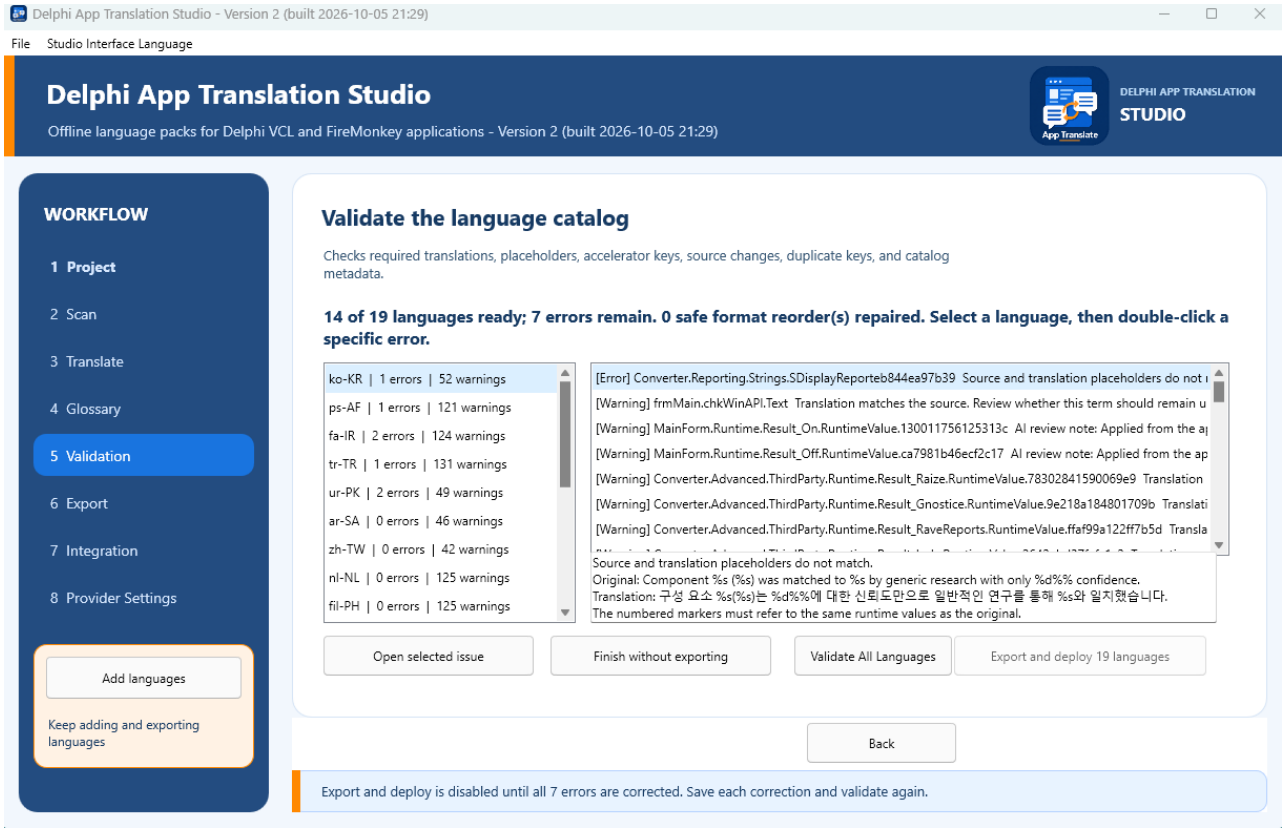


Saved catalogs remain available after a review. Adding a language, reviewing saved text, validating, and leaving drafts are separate choices.

Validation and pack construction

Validation checks the catalog before export. It requires application and language identity, locale settings, and translated text for eligible entries. It checks duplicate keys, source changes, and formatting placeholders. For example, a translated format string must refer to the same runtime arguments as its source; a misplaced numbered marker can insert the wrong value or fail at runtime. Accelerator marker differences, unchanged wording, ambiguous context, low-confidence drafts, and unconfirmed runtime wiring are reported for review at their appropriate severity.

An error prevents export of that language. Warnings do not automatically block it. In a batch, ready languages can be exported while languages with errors remain editable drafts. Export revalidates before writing a pack, so a later catalog change cannot bypass the check merely because the Validation screen was previously clean.



Validation identifies the affected language and entry. The example shows an error state; correct the catalog entry and validate again.

Each runtime pack includes application ID, framework, application version, source language, target language, locale details, translated strings, and a checksum of the source catalog. The source-language pack travels with target packs. The checksum lets the runtime detect a target pack built for a different set of source strings. If a later scan only adds source entries, an older target pack may still be accepted when its shared source strings match; missing entries then fall back to source text. Changed shared source text is not treated as compatible.

The component kit and Delphi project

The Integration page prepares a project-local dependency and a design-time package for the project's VCL or FireMonkey framework. The developer installs the exact generated design BPL through Component > Install Packages > Add. The package registers the corresponding language manager and language selector in the RAD Studio component palette. The manager and selector are placed on the primary form in the IDE, where their properties remain visible in the Object Inspector.

The developer also adds $(PROJECTDIR)\dependencies\DelphiAppTranslation\source to the Delphi Compiler Search path for the configurations and platforms to distribute. When the Studio starts a build, it supplies that path temporarily. It does not make a permanent change to the developer-owned DPROJ setting. This is why a Studio build can succeed while a later independent RAD Studio build still needs its Search path set.

For VCL, the components are TDATVCLLanguageManager and TDATVCLLanguageComboBox. FireMonkey uses TDATFMXLanguageManager and TDATFMXLanguageComboBox. The manager's ApplicationId must match the pack, SourceLanguage must match the saved project's source language, and LanguagesFolder normally points to Localization\Languages. The selector's LanguageManager property connects it to the manager. These are identity and wiring requirements, not cosmetic options.

What happens when the application runs

The runtime discovers packs in the configured LanguagesFolder and verifies that they belong to the application and framework. When loading a target, it also checks the target against the installed source pack: source language, application version, framework, and source-catalog compatibility must agree. An incompatible pack is not offered as a valid language. Missing translated keys fall back to the original source text rather than displaying an empty label.

On first use, the manager can choose a shipped pack matching the operating system language. After the user selects a language, that saved preference takes priority on later launches. The default preference location is the user's local application data; other locations are exposed as component properties. The manager tracks which open forms it has translated, applies a language change to managed forms, and can translate forms created later. It preserves control state by default while changing text.

VCL and FireMonkey have separate form applicators because their control and lifecycle models differ. VCL watches open and newly active forms. FireMonkey also handles its form lifecycle and supported browser content. A control's automatic designer translation is distinct from application-created messages: code that builds text at runtime may need DATTranslateText, DATFormatText, or the corresponding manager method. The runtime remains offline; those calls read the active pack and return source text when no applicable translation is found.

Layout and deployment boundaries

Localization Review can propose width, height, wrapping, or other layout changes for a target language. A proposal is evidence for inspection, not proof that a form will look right on every screen. Pack export includes supported layout rules, and the runtime applies only rules it recognizes as safe for the control. Long labels, right-to-left forms, dialogs, and browser content still require a test of the running application.

Deployment prepares the validated source and target set, copies it to a staging directory, checks the staged files, and then promotes that directory to Localization\Languages. The previous deployed directory is kept as the immediate replacement fallback during promotion. This reduces the chance of leaving a half-copied language set if a copy fails. Export updates the project-local language folder and selected executable folders; an archived portable export is separate and is not silently revised.

The Setup Wizard's automatic build path uses Win32 Release as a baseline when supported, and refreshes other supported Windows configurations when their output folders show they are in use. A manually named application folder can receive packs without being mistaken for a build output. Executable

replacement has its own authorization. Always read the destination list before export: a successful export to one folder does not imply that every copy of the EXE on the computer was updated.

Backup and recovery responsibilities

Before final Wizard processing, the Studio creates a timestamped ZIP Safety Backup of the target project. Temporary WebView2 browser-cache directories are skipped. The ZIP is reopened and checked for file count and the project file before processing continues. This protects the pre-processing project state; it does not replace version control or a release archive.

Recover from Safety Backup is available from the start screen. It previews a selected Studio ZIP and restores into a new empty directory, so the original project is not overwritten. Use a restored project copy when project files need recovery. For an ordinary wording correction, keep the existing project, edit its catalog, validate, export, and test again.

Engineering release check

A release check crosses each boundary in order. Confirm that a saved Delphi project scans as expected; the relevant catalogs are reviewed; each language validates and exports; the intended project and executable folders contain matching source and target packs; and an independent RAD Studio build resolves the project-local dependency. Then run the exact executable to be distributed and switch languages on real forms. If a catalog looks correct but the application does not, inspect runtime wiring, pack identity, and the executable's adjacent Localization\Languages folder before retranslating text.